RAG Fundamentals

Document ID	DOC001
Project	RAG
Category	RAG
Batch	batch_1

Purpose: Original educational notes created as a searchable PDF corpus for a portfolio-scale incremental RAG pipeline. The material is designed to contain meaningful technical sections that naturally produce multiple retrieval chunks.

What Retrieval-Augmented Generation Is

Retrieval-Augmented Generation, usually called RAG, combines information retrieval with a generative language model. Instead of asking a language model to answer only from knowledge encoded during training, a RAG system first searches an external knowledge collection for information related to the user's question. The retrieved passages are then supplied to the language model as context. This makes RAG useful when answers need to reflect private, domain-specific, or frequently changing information.

Practical note. In practice, the implementation should keep ingestion separate from querying, preserve source metadata, and inspect retrieved passages during testing rather than judging the final answer alone.

Core RAG Flow

A typical RAG flow starts with document ingestion. Documents are extracted into text, cleaned, split into chunks, converted into vector embeddings, and stored in a vector database. At query time, the user's question is also embedded. Similarity search compares the question vector with stored chunk vectors and returns the top matching chunks. These chunks are assembled into context and sent with the question to the language model, which produces an answer grounded in the retrieved information.

Practical note. In practice, the implementation should keep ingestion separate from querying, preserve source metadata, and inspect retrieved passages during testing rather than judging the final answer alone.

Why Embeddings Are Needed

An embedding is a numerical vector representing semantic information in text. Texts with related meanings tend to have vectors that are closer in the embedding space even when they use different words. This allows semantic retrieval to find passages based on meaning rather than exact keyword matches. The same embedding model should normally be used for stored chunks and incoming questions so that their vectors exist in a compatible space.

Practical note. In practice, the implementation should keep ingestion separate from querying, preserve source metadata, and inspect retrieved passages during testing rather than judging the final answer alone.

Chunking and Overlap

Long documents are usually divided into smaller chunks before embedding because embedding and generation models have context limits and because retrieval works better when each stored unit is focused. A chunk that is too large can mix unrelated topics; a chunk that is too small can lose important context. Overlap repeats a small portion of neighboring chunks so information near a boundary is less likely to be separated. Chunk size and overlap should be selected based on document structure, model limits, and retrieval quality.

Practical note. In practice, the implementation should keep ingestion separate from querying, preserve source metadata, and inspect retrieved passages during testing rather than judging the final answer alone.

Top-K Retrieval and Grounding

Top-k retrieval means returning the k most relevant chunks for a question. A small k keeps context focused, while a very large k can introduce irrelevant material and consume model context. Grounded generation means instructing the language model to answer using retrieved context and to avoid unsupported claims. Source metadata should be retained so users can trace an answer back to the chunks and documents that supported it.

Practical note. In practice, the implementation should keep ingestion separate from querying, preserve source metadata, and inspect retrieved passages during testing rather than judging the final answer alone.

Metadata Filtering

Vector similarity can be combined with metadata filters. For example, a collection containing documentation from multiple technologies can restrict retrieval to category RAG or project Azure Data Factory before ranking by similarity. Filtering is useful when the user already knows the relevant domain, tenant, product, date range, or document type. Metadata should reflect real properties of the source rather than invented labels added only for demonstrations.

Practical note. In practice, the implementation should keep ingestion separate from querying, preserve source metadata, and inspect retrieved passages during testing rather than judging the final answer alone.

End of document.